

Notas: CoreCLR vs Mono en .NET MAUI

Notas para post: CoreCLR vs Mono en .NET MAUI

Estas son notas de trabajo para armar un post técnico sobre el impacto del runtime elegido en el peso del APK de release en .NET MAUI Android.

Nota 1 — ¿Por qué el APK con CoreCLR pesa más que con Mono?

Contexto

Al compilar una app .NET MAUI para Android en modo **Release**, se puede elegir entre dos runtimes:

- **Mono** — el runtime histórico de Xamarin/MAUI, optimizado para mobile
- **CoreCLR** — el runtime moderno de .NET (desktop/server), portado a mobile más recientemente

Razones del mayor tamaño con CoreCLR

1. Runtime más pesado por diseño

CoreCLR fue construido para desktop/server y luego portado a mobile. Mono, en cambio, fue diseñado desde el inicio para dispositivos embebidos y móviles, siendo mucho más compacto.

2. AOT compilation genera .so más grandes

Con CoreCLR, el compilador AOT (Ahead-of-Time) produce archivos nativos (.so) de mayor tamaño. Mono usa un modelo de AOT más compacto y en algunos casos interpreta parte del IL.

3. Trimming menos maduro en mobile

El linker/trimmer no es igual de agresivo con CoreCLR que con Mono. Mono lleva años con optimizaciones de tamaño específicas para mobile; CoreCLR recién está madurando en ese aspecto.

4. Mayor porción del BCL empaquetada por defecto

CoreCLR tiende a incluir más del Base Class Library en el bundle, aunque esto se puede reducir con trimming agresivo.

Cómo reducir el tamaño con CoreCLR

```
<!-- Activar trimming agresivo -->
<PublishTrimmed>true</PublishTrimmed>
<TrimMode>link</TrimMode>

<!-- Separar APKs por ABI para no empaquetar todas las arquitecturas en uno -->
<AndroidCreatePackagePerAbi>true</AndroidCreatePackagePerAbi>

<!-- Comprimir recursos con AAPT2 (ya está activo por defecto en MAUI) -->
<AndroidUseAapt2>true</AndroidUseAapt2>

<!-- Habilitar compresión de assemblies -->
<AndroidEnableAssemblyCompression>true</AndroidEnableAssemblyCompression>
```

Conclusión

CoreCLR **prioriza rendimiento sobre tamaño**. El mayor peso del APK es un trade-off esperado y documentado. Microsoft está trabajando activamente en reducirlo en versiones futuras del SDK.

Nota 2 — Benchmark: Lista virtual con 10.000 items

Qué se implementó

Una página de benchmark (BenchmarkListPage) en la app de sample, accesible desde el menú hamburguesa (sección **Benchmarks**, primera en la lista).

El benchmark carga 10.000 items parseados desde un JSON pre-generado (~1.5MB) y mide 4 fases por separado:

Fase	Operación
1. JSON deserialize	<code>JsonSerializer.Deserialize<List<BenchmarkRawDto>>()</code>
2. LINQ map + sort	<code>Proyección DTO → modelo + OrderByDescending(score)</code>
3. Date parse + format	<code>DateTime.Parse + .ToString("MMM dd, yyyy") × 10k</code>
4. Bind collection	<code>new ObservableCollection<BenchmarkItem>(list)</code>

El JSON se pre-genera al entrar a la pantalla (no se cuenta en las métricas). El botón **Reload** repite el ciclo completo para obtener múltiples muestras.

Por qué estas fases muestran diferencias entre runtimes

- **JSON deserialize:** `System.Text.Json` se beneficia del JIT de CoreCLR en code paths repetitivos (reflection-free source gen aparte).
- **LINQ + proyección:** El compilador JIT de CoreCLR optimiza mejor los iteradores y lambdas en caliente.
- **Date parse/format:** Operaciones de string intensivas donde el GC y la inlining de CoreCLR marcan diferencia.
- **Bind collection:** Mide el overhead del constructor de `ObservableCollection` y el sistema de notificaciones.

Estructura de cada item (template rico)

- Badge circular con score (color rojo/naranja/verde según valor)
- Nombre + email + tags
- Departamento + fecha formateada + estado activo/inactivo

Archivos involucrados

- `Models/BenchmarkItem.cs` — modelo de presentación
- `Models/BenchmarkRawDto.cs` — DTO para deserialización JSON
- `ViewModels/BenchmarkListViewModel.cs` — lógica de benchmark
- `Pages/BenchmarkListPage.xaml + .xaml.cs` — UI con `CollectionView` sin `ScrollView` anidado (virtualización activa)

Nota: además del bind inicial, esta página también mide el rendimiento durante el **scroll de la lista**. Ver Nota 5 para el detalle de esas métricas.

Nota 3 — CoreCLR es estricto con la coerción de tipos en bindings

El problema

En .NET MAUI, cuando usás `x:DataType` en un `DataTemplate` (bindings compilados), el compilador XAML genera código con asignaciones directas entre la propiedad del `ViewModel` y la propiedad del control.

Ejemplo que falla en CoreCLR pero funciona en Mono:

```
<DataTemplate x:DataType="models:BenchmarkItem">
  <!-- Score es int, Text espera string -->
  <Label Text="{Binding Score}" />
</DataTemplate>
```

- **Mono:** el runtime hace la conversión `int` → `string` implícitamente a través de `BindingExpression`. No explota.
- **CoreCLR:** el binding compilado intenta asignar el `int` directamente al `string`, lanza `TargetInvocationException` envolviendo un `InvalidCastException`.

Por qué ocurre

Con bindings compilados, el XAML genera algo parecido a:

```
label.Text = item.Score; // int → string: InvalidCastException en CoreCLR
```

Mono tenía una capa de reflexión más permisiva que coercionaba tipos primitivos. CoreCLR con bindings compilados es estricto: los tipos deben coincidir exactamente.

Síntoma en logcat

```
android.runtime.JavaProxyThrowable: [System.Reflection.TargetInvocationException]:
Exception has been thrown by the target of an invocation.
at BindingExpression+BindingExpressionPart.TryGetValue
at ...BenchmarkListPage+<c.<InitializeComponent>b__1_1
```

El inner exception real (`InvalidCastException`) no aparece en el logcat de Android — solo el wrapper. Esto lo hace difícil de diagnosticar.

Fix

Dos opciones:

```
<!-- Opción A: StringFormat en el binding -->
<Label Text="{Binding Score, StringFormat='{0}'}" />

<!-- Opción B (recomendada): exponer la propiedad como string desde el ViewModel -->
<Label Text="{Binding ScoreText}" />
```

La opción B es preferible porque el problema no está solo en XAML — cualquier lugar que use reflexión para asignar el valor podría fallar igual.

Regla general para CoreCLR

En CoreCLR con bindings compilados (x:DataType), **el tipo de la propiedad del modelo debe coincidir exactamente con el tipo de la propiedad del control** al que se bindea. No asumir coerción implícita de tipos primitivos.

Tipos que suelen generar este problema: - int / double / float → string (ej: Label.Text) - Color struct → cualquier propiedad que espere Color pero viene de reflexión - bool → string en controles que no tienen un BoolToStringConverter nativo

Nota 4 — Benchmark: Animación de partículas con SkiaSharp

Qué se implementó

Una página de benchmark (BenchmarkSkiaPage) que renderiza N partículas animadas con trail/estela sobre un SKCanvasView fullscreen usando SkiaSharp. Cada frame mide por separado el tiempo de física y el tiempo de draw.

Configuración del benchmark

- **Trail:** 20 posiciones históricas por partícula, dibujadas con alpha decreciente
- **Niveles de carga:** 50 / 100 / 200 / 500 partículas
- **Draw calls por frame:** N partículas × 21 círculos (20 trail + 1 principal)
 - Ej: 200 partículas → 4.200 draw calls/frame → ~250k draw calls/segundo a 60fps
- **Loop:** InvalidateSurface() al final de cada PaintSurface → rendering continuo sin timer externo

Métricas que muestra en tiempo real

Métrica	Descripción	Por qué importa para la comparación
FPS	Frames por segundo del frame actual (1 / tiempo_del_último_frame)	Referencia instantánea, muy variable
avg FPS	Promedio móvil de los últimos 120 frames (~2 segundos)	El número más representativo del rendimiento sostenido
min FPS	Mínimo histórico desde que se inició la animación	Revela GC pauses y JIT stalls — el peor caso que ve el usuario
physics ms	Tiempo del loop C# por frame: actualizar posiciones, rebotar en bordes, enqueue/dequeue del trail	Mide el rendimiento del JIT sobre código numérico puro
draw ms	Tiempo de las llamadas a Skia canvas por frame: N × 21 DrawCircle() con paint/alpha	Mide el overhead del interop .NET → Skia nativo

Cómo comparar CoreCLR vs Mono

Ambas versiones conviven en el mismo dispositivo con app IDs distintos:

```
# Compilar CoreCLR
dotnet build -f net11.0-android -c Release -p:UseMono=false
```

```
# Compile Mono
```

```
dotnet build -f net11.0-android -c Release -p:UseMono=true
```

Protocolo de medición: 1. Abrir la app, ir a **Benchmarks** → **Skia particles animation** 2. Seleccionar nivel de carga (50 / 100 / 200 / 500) 3. Tocar **Start** y esperar ~30 segundos para que el JIT se estabilice 4. Anotar: avg FPS, min FPS, physics ms, draw ms 5. Repetir en la otra app (mismo dispositivo, mismas condiciones)

Tabla de resultados (completar)

100 partículas

Métrica CoreCLR Mono

avg FPS	—	—
min FPS	—	—
physics ms	—	—
draw ms	—	—

200 partículas

Métrica CoreCLR Mono

avg FPS	—	—
min FPS	—	—
physics ms	—	—
draw ms	—	—

500 partículas

Métrica CoreCLR Mono

avg FPS	—	—
min FPS	—	—
physics ms	—	—
draw ms	—	—

Qué se espera observar

- **physics ms:** CoreCLR más rápido por JIT agresivo en loops numéricos y mejor inlining
- **min FPS:** Mono más estable (GC conservador, menos spikes); CoreCLR puede tener caídas por GC generacional
- **draw ms:** Similar en ambos — Skia corre en código nativo, el interop es equivalente
- **avg FPS:** Depende del dispositivo; a 200+ partículas debería verse diferencia

Archivos involucrados

- Models/Particle.cs — posición, velocidad, radio, color, Queue<SKPoint> trail cap 20
- ViewModels/BenchmarkSkiaViewModel.cs — loop en PaintSurface, stats cada 10 frames
- Pages/BenchmarkSkiaPage.xaml + .xaml.cs — SKCanvasView fullscreen + header stats

☐ Mencionar el impacto de usar AAB (Android App Bundle) vs APK para distribución en Play Store

- ☐ Comparar rendimiento en runtime: ¿vale la pena el peso extra de CoreCLR?
- ☐ Hablar de cuándo conviene cada runtime según el tipo de app
- ☐ Referencias: docs oficiales de Microsoft sobre runtimes en MAUI

Nota 5 — Benchmark: Scroll FPS en lista virtualizada de 10.000 items

Qué se mide

Luego de que la lista termina de bindear los 10.000 items, la página dispara automáticamente:

1. Un scroll animado hasta el último item (forzando la virtualización de todo el viewport)
2. Una ventana de medición de 5 segundos con un `IDispatcherTimer` a ~16 ms (60 fps objetivo)
3. Captura de heap managed antes y después del scroll

Métricas nuevas en el stats panel

#	Métrica	Descripción	Por qué importa
5	Avg FPS (scroll)	FPS promedio calculado como $1000 / \text{avg_tick_delta}$ durante los 5s de scroll	Rendimiento sostenido de la UI bajo carga de virtualización
6	Min FPS (scroll)	FPS mínimo calculado como $1000 / \text{max_tick_delta}$	Revela drops por GC, JIT stalls o item realization costosa
7	Scroll duration	Duración total de la ventana de medición (ms)	Referencia temporal
8	Managed heap Δ	<code>GC.GetTotalMemory(false)</code> después – antes del scroll (KB)	Cuánto heap nuevo se asignó para realizar los item templates durante el scroll

Cómo funciona el FPS timer

Se usa `IDispatcherTimer` (main thread) con intervalo de 16 ms. Cada tick registra el timestamp (`Stopwatch.ElapsedMilliseconds`). Al terminar la ventana se calculan los deltas entre ticks consecutivos:

```
avg_fps = 1000 / mean(deltas)
min_fps = 1000 / max(delta)  ← el delta más largo = peor frame
```

Este método mide la **tasa de dispatch del hilo principal** — si el UI thread está congestionado (virtualizando items, ejecutando bindings), los ticks se retrasan y el FPS baja. Es un proxy confiable para comparar CoreCLR vs Mono bajo carga de scroll.

Protocolo de medición

1. Abrir la app → **Benchmarks** → **Virtual list (10k items)**
2. Tocar **Reload benchmark** y esperar que cargue
3. Tras ~800 ms, la lista hace scroll automático al final — **no tocar el dispositivo**
4. Esperar 5 segundos a que aparezcan los valores de scroll
5. Anotar los 4 valores de la sección “Scroll FPS (auto)”

6. Repetir en la otra app (mismo dispositivo, mismas condiciones)

Tabla de resultados (completar)

Métrica	CoreCLR	Mono
Avg FPS (scroll)	—	—
Min FPS (scroll)	—	—
Scroll duration	—	—
Managed heap Δ	—	—

Qué se espera observar

- **Avg FPS:** CoreCLR debería ser igual o mejor — el JIT optimiza mejor los loops de binding
- **Min FPS:** Mono puede ser más estable (GC menos agresivo, menos spikes); CoreCLR puede mostrar drops si el GC generacional barre durante el scroll
- **Managed heap Δ :** Similar en ambos — el delta refleja la realización de item templates, no el runtime en sí; diferencias marcan colecciones temporales más grandes en uno de los dos

Archivos involucrados (cambios de esta nota)

- ViewModels/BenchmarkListViewModel.cs — nuevas propiedades + StartScrollMeasurement / StopScrollMeasurement / timer FPS
- Pages/BenchmarkListPage.xaml.cs — suscripción a PropertyChanged, trigger de scroll, ventana de 5s con CancellationToken
- Pages/BenchmarkListPage.xaml — 4 nuevas filas en el stats card + ActivityIndicator de estado

Nota 6 — Benchmark: Tiempo de navegación entre páginas (Navigation Timing)

Qué se mide

El tiempo que tarda la app en ir desde que el usuario toca un ítem del menú (Main) hasta que la página de destino está **completamente lista** para mostrarse al usuario. Cubre: ejecución de `Shell.GoToAsync`, construcción del ViewModel y la Page (DI), y — en páginas con datos asíncronos — la carga y binding inicial.

Cómo funciona

El sistema usa `NavigationTimer` (habilitado con la constante de compilación `ENABLE_NAV_TIMING`):

Momento	Código	Qué ocurre
T0 — arranque	<code>BaseViewModel.GoToAsync</code>	Justo antes de llamar a <code>Shell.Current.GoToAsync</code> , se llama a <code>NavigationTimer.Start(route)</code> y arranca el Stopwatch.
T1 — página lista	<code>BaseViewModel.ReportPageReady()</code>	Llama a <code>NavigationTimer.Complete(Title)</code> , detiene el reloj y dispara el evento <code>NavTimingCompleted</code> .

`ReportPageReady()` se invoca de dos formas según la página: - **Automática:** `BaseContentPage.OnAppearing()` la llama si la página no carga datos async (el caso simple). - **Manual:** las páginas que cargan datos en forma asíncrona deben llamar a `vm.ReportPageReady()` ellas mismas una vez que el binding esté completo (por ejemplo, después del `await` que trae los datos).

Cómo se muestra el resultado

Al completarse la navegación, aparece un badge flotante (`NavTimingOverlay`) superpuesto sobre la página destino con el formato:

🕒 <Título de la página> – <ms> ms

El badge desaparece solo a los 10 segundos o al tocarlo. Si se navega antes, se descarta automáticamente.

Cómo habilitar / deshabilitar

Por defecto está activo en Debug y Release. Se puede apagar pasando `-p:EnableNavTiming=false` al compilar:

```
dotnet build -f net11.0-android -c Release -p:EnableNavTiming=false
```

Caso a probar

1. Abrir la app → pantalla **Main**
2. Tocar cualquier ítem del menú lateral (ej: **Button**, **TextField**, **Progress indicator**)
3. Observar el badge 🕒 que aparece en la esquina de la página destino
4. Anotar el tiempo (ms) para cada página
5. Repetir en CoreCLR y Mono para comparar: la diferencia refleja el costo de construcción de la página + bindings en cada runtime

Qué se espera observar

- Páginas simples (sin carga async): <100 ms en ambos runtimes; CoreCLR puede ser marginalmente más rápido por JIT
- Páginas con listas grandes (ej: **BenchmarkList**): la diferencia se amplifica porque el binding de la colección también está en el camino crítico
- En el primer acceso a cada página, CoreCLR puede tardar más (JIT en frío); en accesos subsiguientes tiende a nivelar o superar a Mono

Archivos involucrados

- `Utils/NavigationTimer.cs` — lógica de medición (stopwatch + evento `NavTimingCompleted`)
- `ViewModels/BaseViewModel.cs` — `GoToAsync (T0)` y `ReportPageReady (T1)`
- `Pages/BaseContentPage.cs` — llama a `ReportPageReady` en `OnAppearing` para páginas simples
- `Views/NavTimingOverlay.xaml` + `.xaml.cs` — badge flotante que muestra el resultado durante 10 s
- `AppShell.xaml.cs` — `EnsureNavTimingOverlay()` inyecta el overlay en el Grid de cada página

Última actualización: 2026-07-09 — agregada Nota 6 (navigation timing Main → página + `NavTimingOverlay`)